

I cannot access the actual SBC content from that `share.google` shortlink because it redirects into the Google app and the board files are not publicly visible from here. If you can upload:

- SBC schematic PDF
- DDR section screenshot
- BOM
- Board photo
- DDR chip markings

then I can give you an exact board-specific DDR3L validation strategy.

But since your processor is the **NXP T2080**, I can still tell you the **professional DDR3L testing flow used on embedded SBCs**.

DDR3L Testing in an Embedded SBC (T2080 Based)

Testing DDR is done in layers:

1. Power validation
2. Clock validation
3. Reset/initialization
4. Training verification
5. Address/data integrity
6. Stress testing
7. ECC validation
8. Signal integrity validation
9. Thermal/reliability testing

This is exactly how board bring-up teams work.

1. FIRST: Understand the DDR Path in Your SBC

In your board:

```
graph TD; A[T2080 DDR Controller] --> B[DDR PHY]; B --> C[PCB traces]; C --> D[DDR3L chips];
```

Failures can happen at:

- controller config
- PHY timing
- routing
- power

- SI/PI
 - DRAM chip
 - soldering
 - timing setup
-

2. Step-By-Step DDR Bring-Up Validation

STEP 1 — Visual Inspection

Before software testing.

Check:

- DDR chip solder quality
- missing resistors
- damaged termination resistors
- power rails
- clock oscillator

Common failures:

- cold solder joints
 - missing VTT termination
 - swapped byte lanes
-

STEP 2 — Power Rail Validation

DDR3L uses:

Rail	Typical
VDD	1.35V
VPP	sometimes 2.5V
VTT	VDD/2
VREF	VDD/2 reference

Using oscilloscope:

Check:

- ripple
- sequencing
- stability

Critical:

VTT = ~0.675V

Bad VTT causes:

- intermittent DDR failures

STEP 3 — Clock Validation

Check:

- differential CK/CK#
- frequency
- jitter
- duty cycle

Using oscilloscope:
verify clean clock.

Example:

800 MHz DDR clock
→ DDR1600 effective

STEP 4 — Reset and Initialization

Observe:

- RESET#
- CKE
- CS#

Sequence must follow JEDEC timing.

The T2080 controller initializes:

- mode registers
- ZQ calibration
- timings
- training

If initialization fails:

- memory inaccessible
- boot crashes

3. MOST IMPORTANT PART — DDR Training Validation

Modern DDR absolutely depends on training.

The T2080 PHY performs:

- write leveling
- read leveling
- DQS gate training
- delay calibration

Failures here are VERY common.

3.1 Write Leveling

Purpose:

align DRAM clock with DQS.

Problem:

clock arrives at different times due to PCB routing.

Controller adjusts delays.

Failure symptoms:

- writes fail
 - intermittent corruption
-

3.2 Read Leveling

Aligns incoming DQS with controller sampling clock.

Failure:

- read corruption
 - bit flips
-

3.3 DQS Gate Training

Determines valid capture window.

This is timing critical.

4. SOFTWARE MEMORY TESTING

Now actual memory validation.

4.1 Walking Ones Test

Write:

```
00000001
00000010
00000100
```

Purpose:

- detect stuck-at faults
 - shorted data lines
-

4.2 Walking Zeros

Opposite patterns.

4.3 Checkerboard Pattern

Write:

```
10101010
01010101
```

Purpose:

- detect coupling faults
 - adjacent bit interference
-

4.4 Address Line Test

Very important.

Write:

```
mem[i] = i;
```

Then verify.

Detects:

- address shorts
 - swapped address lines
-

4.5 Random Pattern Testing

Pseudo-random writes/reads.

Better real-world coverage.

4.6 Burst Access Testing

DDR works in bursts.

Test:

- sequential reads
- burst writes
- DMA transfers

Purpose:

- verify timing margins
-

5. CACHE-BYPASS TESTING

VERY IMPORTANT.

Sometimes cache hides DDR failures.

You must:

- disable cache
OR
- use uncached regions

Otherwise:

CPU cache may fake successful DDR tests.

6. DMA-BASED DDR TESTING

Use DMA engine.

This stresses:

- interconnect
- arbitration
- bursts
- refresh overlap

Much more realistic than CPU-only tests.

7. MULTI-CORE STRESS TESTING

T2080 is multicore.

Run:

- simultaneous accesses
- different cores
- different banks

Purpose:

- expose timing/routing weaknesses
-

8. ECC TESTING (If ECC Enabled)

T2080 often supports ECC.

Test:

- inject single-bit error
- verify correction
- inject double-bit error
- verify detection

Expected:

- SECEDED operation
-

9. TEMPERATURE STRESS TESTING

DDR failures often appear ONLY under temperature.

Test:

- cold boot
- hot operation
- continuous stress

Common issue:

timing margin collapses at temperature extremes.

10. REFRESH TESTING

DRAM leaks charge.

Test by:

- reducing refresh frequency

- monitoring retention errors

Purpose:

- identify weak DRAM cells
-

11. LONG-DURATION SOAK TEST

Industry standard.

Run:

- 12 hours
- 24 hours
- 48 hours

Continuous:

- reads/writes
- random traffic
- DMA
- multicore load

Goal:

detect intermittent failures.

12. Signal Integrity (Advanced Hardware Validation)

This is where senior hardware engineers work.

Using high-speed oscilloscope:
check:

Signal	Check
CK	jitter
DQ	eye opening
DQS	timing alignment
Address lines	overshoot/ringing

12.1 Eye Diagram Analysis

Critical DDR validation method.

Good eye:

- wide opening
- stable timing

Bad eye:

- noise
 - jitter
 - reflections
-

12.2 Common SI Problems

Problem	Cause
Ringing	bad termination
Crosstalk	routing issue
Timing failure	skew
Random crashes	marginal eye

13. PCB-Level DDR Validation

Check:

- fly-by topology
- trace length matching
- impedance control
- termination placement

DDR routing is EXTREMELY sensitive.

14. Tools Commonly Used

Software

Tool	Purpose
U-Boot mtest	basic memory test
memtester	Linux DDR stress
stressapptest	aggressive stress

Tool	Purpose
Bare-metal tests	low-level validation

Hardware

Tool	Purpose
Oscilloscope	signal analysis
Logic analyzer	protocol debug
JTAG	low-level access
Thermal chamber	reliability

15. PROFESSIONAL DDR DEBUG FLOW

When DDR fails:

Failure Type A — No Boot

- Possible causes:
- bad timing config
 - power rail issue
 - training failure
 - clock issue
-

Failure Type B — Random Crashes

- Possible:
- SI issue
 - temperature margin
 - refresh issue
 - marginal timing
-

Failure Type C — Specific Bits Fail

Possible:

- damaged DRAM chip
 - bad DQ line
 - solder issue
-

Failure Type D — Large Transfers Fail

Possible:

- burst timing issue
 - DQS alignment issue
-

16. T2080-Specific Things To Study

You MUST study:

- DDR controller registers
- SPD configuration
- timing setup
- PHY training status
- memory map
- ECC registers

From:

- T2080 Reference Manual
 - Board BSP
 - U-Boot DDR init code
-

17. VERY IMPORTANT — How You Can Actually Start on Your Board

Tomorrow do this:

Phase 1 — Identify DDR Part

Physically inspect DDR chip.

Write down marking.

Example:

MT41K256M16



This changes EVERYTHING.

Phase 2 — Find Board DDR Configuration

Determine:

- x8 or x16
 - number of ranks
 - ECC or non-ECC
 - total capacity
 - speed grade
-

Phase 3 — Run Basic Tests

If Linux available:

Install/run:

- memtester
- stressapptest

Example:

```
memtester 512M 10
```

Phase 4 — U-Boot Tests

In U-Boot:

```
mtest
```

or custom DDR diagnostics.

Phase 5 — Observe Failure Patterns

Important:

DDR bugs rarely fail randomly.

Patterns matter.

18. How To Explain DDR Testing to Colleagues

Use this structure:

1. Initialization validation
2. Timing/training validation
3. Functional testing
4. Stress testing
5. ECC verification
6. SI/PI validation
7. Thermal reliability

That sounds exactly like real hardware validation methodology.

19. BEST PRACTICAL LEARNING METHOD

Do NOT just read theory.

Use:

1. DDR datasheet
2. T2080 DDR init code
3. Actual board
4. Oscilloscope captures
5. Memory stress logs

That combination creates real expertise.

20. What I Need From You To Go Deeper

Upload ANY of these:

- DDR chip photo
- schematic
- DDR section PDF
- board photo
- U-Boot logs
- Linux dmesg
- memory map

Then I can help you with:

- exact DDR architecture
- rank/bank mapping
- timing calculations
- DDR controller configuration

- testing scripts
- failure diagnosis
- SI analysis
- training analysis
- ECC validation
- presentation for your scientist/team

The exact DDR chip part number is the key.